

IR remote to hex on seven segment v1

Dennis Junusov

12.09.2026

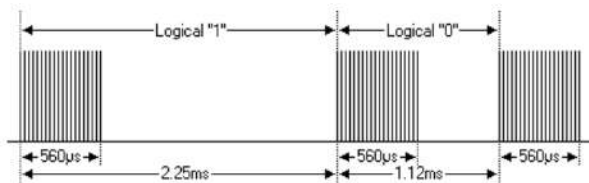
1 Goal

The goal of this experiment is to prepare for a project where I use the data from the infrared remote control to realize a small calculator using the buttons on the IR remote. As I don't have any documentation about the remote control, I don't know what signal/data the remote is sending when pressing a button. So the idea is to create a project where I will display the received data on the 7seg display in hex to find out what the board receives when a button is pressed.

2 Details

- Two digits will be displayed (00 to 0F)
- We have to multiplex between first and second digit
- I assume standard NEC protocol where the data is divided into four bytes
 - Address
 - Address inverted
 - Command
 - Command inverted
- Binary code on the dip switches is from right to left
- Dip switches are used to switch between recieved bytes
 - "0000" → cmd inverted
 - "0001" → cmd
 - "0010" → address inverted
 - "0100" → address

Modulation



The NEC protocol uses pulse distance encoding of the bits. Each pulse is a 560µs long 38kHz carrier burst (about 21 cycles). A logical "1" takes 2.25ms to transmit, while a logical "0" is only half of that, being 1.12ms. The recommended carrier duty-cycle is 1/4 or 1/3.

Protocol

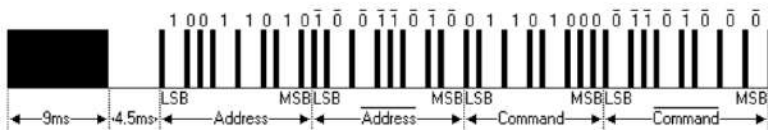


Figure 1: Standard NEC protocol. Source <https://www.sbprojects.net/knowledge/ir/nec.php>



Figure 2: IR remote

3 Results

As the remote is equipped with 21 buttons I will only show a few:

- Button Zero is pressed and the dip switches are 0000 which means "cmd inverted" is being displayed



Figure 3: Data from button Zero (command inverted)

- Button Zero is pressed and the dip switches are 0001 which means "cmd" is being displayed



Figure 4: Data from button Zero (command)

- Button Zero is pressed and the dip switches are 0010 which means "address inverted" is being displayed
- The address is simply "0000 0000" which is apparently expected from cheap IR remotes. Hence the inverse is "1111 1111" $\rightarrow$ "FF".



Figure 5: Data from button Zero (address inverted)

- Button Zero is pressed and the dip switches are 0100 which means "address" is being displayed.



Figure 6: Data from button Zero (address)

- No button is pressed and the dip switches are 1000 which covers the "ELSE" case of the dip switch selection to display the different bytes where I decided to write "1111 1111" to the hex decoder. Hence → "FF"



Figure 7: Invalid data selections

- Following dip switch selection is "0000" and buttons CH+, seven and nine are pressed

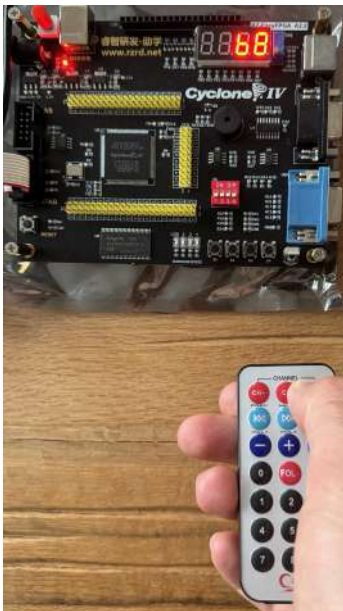


Figure 8: Buttons CH+, seven and nine

- Following are all buttons and their received data. I used a python script (which can also be found in the directory) to compute the inverses which perfectly matched the displayed code on the seven segment display

Table 1: IR Remote Button Hexadecimal Codes and Inverses

Button	Hex Code	Inverse Code (cmd)
CH−	BA	45
CH	B9	46
CH+	B8	47
Left	BB	44
Right	BF	40
Play	BC	43
Minus	F8	07
Plus	EA	15
EQ	F6	09
Zero	E9	16
FOL−	E6	19
FOL+	F2	0D
One	F3	0C
Two	E7	18
Three	A1	5E
Four	EE	11
Five	E3	1C
Six	A5	5A
Seven	BD	42
Eight	5A	A5
Nine	B5	4A